

FRAME IT AS CASES

Work That Speaks for Itself

Name	Omer Yar Khan
Track	Backend AI Engineering
Program	FlyRank AI Internship
Assignment	Week 3 — General AI Fluency: Frame It as Cases

1. Voice Card

Standing instruction added to my AI workspace, applied to every draft below.

VOICE CARD

Direct, precise, plain, no buzzwords, tested.

Applied as a standing prompt: "Use this voice for everything you draft for me. Short sentences. No 'passionate,' 'results-driven,' or 'dynamic.' If a line sounds like a generic AI bio, rewrite it the way I'd actually say it."

2. Sitemap Coverage

The portfolio sitemap calls for four pages: Home / Hero, Work / Case Studies, About, and Contact. This document frames the three pieces that fill Work / Case Studies, plus the bio and contact copy that fill About and Contact.

3. Case Studies

Case 1 — API Response Contract Validator & Drift Detector

The problem. FlyRank-style backend APIs can change shape quietly — a field's type flips, a field disappears, a new one shows up — without breaking a naive check. Manually eyeballing every response against what I remembered the contract to be was slow and unreliable.

What I did (and decided). I built a small Python CLI with two independent checks: schema validation (required fields, types, nullability) and shape-drift detection (infers a response's shape and diffs it against a saved baseline). I decided to keep these two checks separate rather than combined, because a response can pass a loose schema and still have quietly drifted from what it used to look like. I also decided the tool would only accept a relative endpoint path, never a full URL, so it can't be pointed at the wrong host by accident.

What came of it. 22 pytest tests pass, all offline (HTTP calls are mocked). I ran the CLI against a deliberately drifted fixture and it correctly caught a type change (integer → string), a removed field, and a new field — verified end to end, not just written.

What I'd do differently next time: Extend shape inference to check one level into nested objects and arrays, not just the top level.

Case 2 — Portfolio Evidence Tracker (“the Evidence Chain”)

The problem. It's easy to claim I automated something and saved time. It's harder to prove it, and easy to accidentally mark something “done” before it's actually backed by evidence — including the one step no automation should ever skip: a human actually checking it.

What I did (and decided). I designed a FastAPI + SQLAlchemy backend around one core idea: a Task only becomes “portfolio-ready” once seven things exist in order — Task, Input, Process, Output, Measurement, Human Verification, Case Study. I decided the measurement engine should never invent a number: if timing data's missing, it returns “Measurement not yet verified” instead of guessing. I also decided that two decisions — is this evidence honest, and is this my strongest work — get their own data model that no code path can auto-complete; only I can mark them decided.

What came of it. 33 pytest tests pass. I ran the live server end to end and confirmed a task with everything else in place — input, process, output, even a real measurement — still correctly reported “Not portfolio-ready” until I explicitly marked it human-verified.

What I'd do differently next time: Add a real authentication layer before this leaves local use — right now it assumes a single trusted user.

Case 3 — To-Do CRUD API

The problem. Before building anything more complex, I needed a clean, minimal reference for REST fundamentals — correct status codes, real validation, and clear docs — the kind of foundation the other two pieces build on.

What I did (and decided). I built a small in-memory FastAPI CRUD API to a fixed spec. I decided to keep state in a plain Python list on purpose, no database, so the focus stayed on getting the edge cases right: empty title, missing ID, an update call with neither field set.

What came of it. Tested every endpoint against a running server with real requests — confirmed 200, 201, 204, 400, and 404 responses for every case in the spec, including the edge cases.

What I'd do differently next time: Nothing structural — for its stated scope, this one's finished as-is.

4. Bio & Contact / CTA Copy

BIO

I'm Omer, a backend engineering intern at FlyRank. I build small, tested Python tools — API contract checks, evidence-tracking systems, CRUD APIs — that turn repetitive backend work into something measurable and repeatable.

CONTACT / CTA

Want something similar built for your workflow? Email me and I'll walk you through what I built and why.

5. Before / After: Generic AI Line vs. Edited

BEFORE — GENERIC AI OUTPUT

"Leveraged cutting-edge AI-powered automation to deliver scalable, production-ready solutions driving measurable business impact."

AFTER — MY EDITED VERSION

"I built three small backend tools this internship, tested every one, and can show you exactly what still isn't verified."

6. Reflection

The interview step was the actual work here — not the writing. Answering "what did you decide and why" one question at a time surfaced details a "describe my project" prompt would have skipped entirely: why schema validation and drift detection stayed as two separate checks, why the measurement engine refuses to guess a number, why Just-Me decisions needed their own model. The voice card mattered more once I had a first AI draft to compare it against — the before/after above is the clearest evidence of what it actually caught.

7. Submission Checklist

- ✓ Voice card written and added as a standing instruction
- ✓ All three pieces the sitemap calls for are framed as cases
- ✓ Each case has the three beats: problem, what I did, what came of it
- ✓ Bio and contact/CTA copy drafted in the same voice

- ✓ One before/after included: generic AI line vs. edited version
- ✓ Every line read aloud and edited — nothing left I wouldn't say or can't stand behind